Data Warehousing

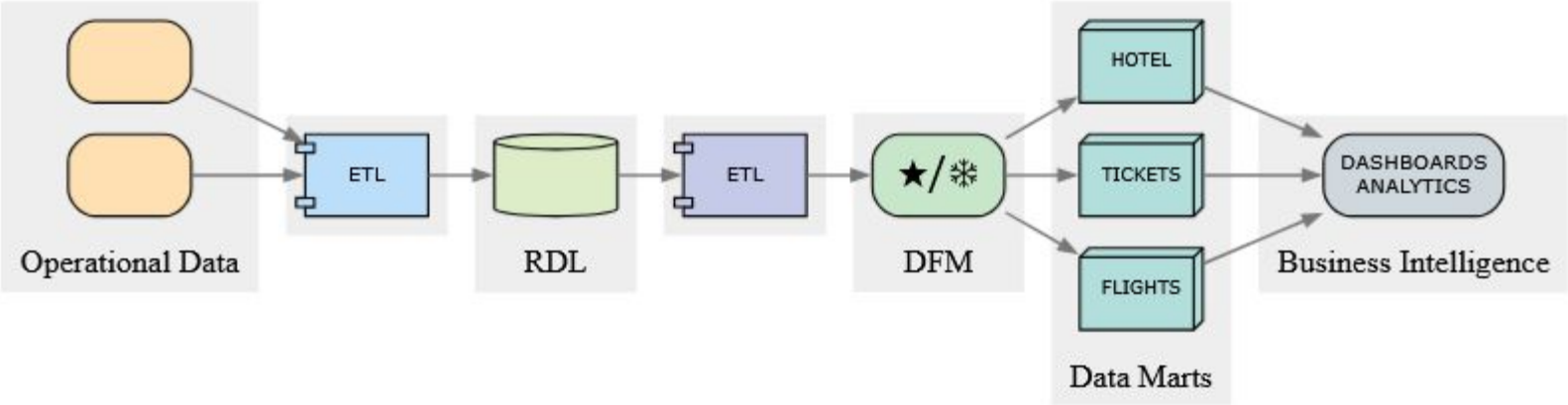
Vlad Turno (1835365)

Data Warehouse

Data processing pipeline divided into four main steps:

- Raw data extraction, cleansing, filtering, uniformation & reconciliation
- Multidimensional fact modeling and star/snowflake schematization
- Reconciled data transformation and loading into primary warehouse
- Data marts subdivision/aggregation and business intelligence tools integration

Three - Layer Architecture



Data Sources

- Operational data from different corporations
  - All of them being in the tour operator field
  - All of them having similar infrastructure
- Almost identical backend scope & organization
  - Flight & Hotel booking, tickets, reservations
  - Tour operators, clients and packages management
  - Managing of promotions and billing/accounting
- Few decades of activity, few million of rows (not that much but enough to make meaningful analysis with statistical relevance)

Operational Data

- Backend schema a bit messy and (sometimes) poorly designed
  - Continuous request for new features over a long timespan
  - Unclear system requirements during development iterations
  - Tables with (too) many fields having cryptic meaning and doubtful utility
- Corporation's (understandable) limitations & guidelines
  - Personal data anonymization for every guest (or passenger) reported within any reservation (or ticket)
  - Prohibition on taking payment & invoicing data outside the company database

ETL Queries

```sql
WITH CitiesOrdered AS (
    SELECT
        CAST(t.Tkt_id AS NVARCHAR(50)) AS Tkt_id,
        CAST(v2.city AS NVARCHAR(3)) AS city,
        ROW_NUMBER() OVER (
            PARTITION BY t.Tkt_id
            ORDER BY v2.ord
        ) AS city_order
    FROM meridiano.dbo.tkt t
    INNER JOIN meridiano.dbo.vettori v ON t.Tkt_vettore = v.vett
    CROSS APPLY (VALUES
        (1, t.Tkt_Cit1),
        (2, t.Tkt_Cit2),
        (3, t.Tkt_Cit3),
        (4, t.Tkt_Cit4),
        (5, t.Tkt_Cit5)
    ) v2(ord, city)
    WHERE v2.city IS NOT NULL AND LTRIM(RTRIM(v2.city
)
INSERT INTO reconcileddatalayer.dbo.segments (
    flight,
    segment,
    departure,
    arrival,
    src
)
SELECT
    a.Tkt_id,
    a.city_order,
    a.city,
    b.city,
    'Meridiano'
FROM CitiesOrdered a
JOIN CitiesOrdered b
    ON a.Tkt_id = b.Tkt_id
    AND a.city_order = b.city_order - 1
ORDER BY a.Tkt_id, a.city_order;
```

```sql
insert into reconcileddatalayer.dbo.airlines
select
    cast(mrd.g_air_code as nvarchar(3)),
    cast(mrd.g_air_name as nvarchar(50)),
    0
from meridiano.dbo.g_airlines as mrd
where not exists (
    select 1
    from reconcileddatalayer.dbo.airlines as
    where rdl.code = mrd.g_air_code );
```

```sql
ert into reconcileddatalayer.dbo.facilities (id, service, de
ect m.servizit_id,
 cast(m.servizit_servizi as nvarchar(5)),
 m.servizit_desc,
 m.servizit_forn,
 m.servizit_costoe,
```

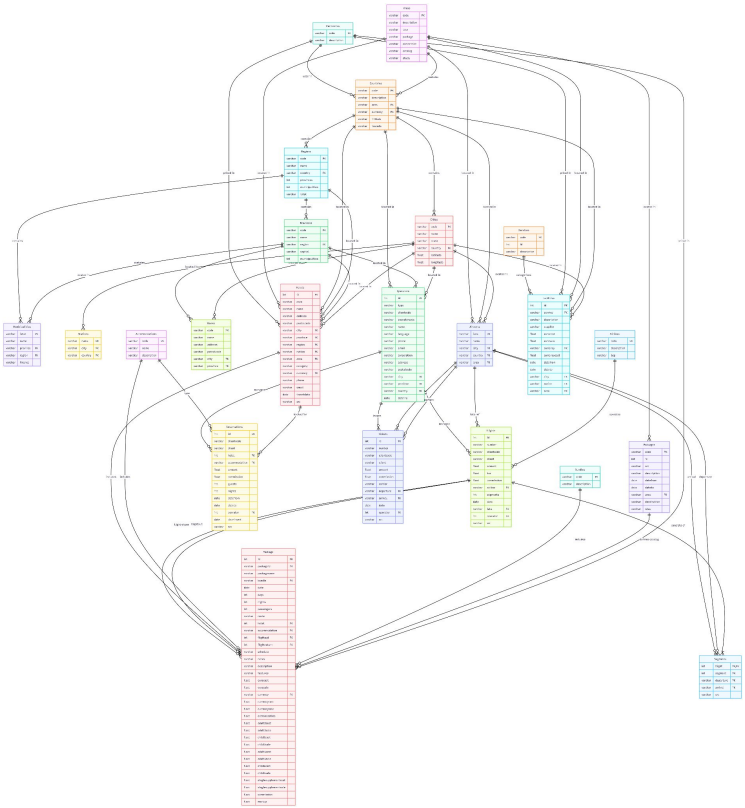
```sql
insert into ReconciledDataLayer.dbo.Hotels (id, code, name
select
    cast(h.hotel_key as nvarchar(50)),
    h.hotel_des,
    h.hotel_ind,
    case when h.hotel_cap = '' then NULL else cast(h.hotel
    cast(h.hotel_cit as nvarchar(3)),
    case when h.hotel_pr = '' then NULL else cast(h.hotel_
    case when h.hotel_regione = '' then NULL else cast(h.hotel_regione as nvarchar(3)) end,
    cast(h.hotel_naz as nvarchar(3)),
    cast(h.hotel_area as nvarchar(3)),
    case when (h.hotel_cat = '' or h.hotel_cat is not NULL) then NULL else cast(h.hotel_cat as nvarchar(7)) end,
    case when h.hotel_val = '' then NULL else cast(h.hotel_val as nvarchar(3)) end,
    case when not (h.hotel_tel = '' or h.hotel_tel is not NULL) then cast(h.hotel_tel as nvarchar(13)) else case when (h.ho
    case when (h.hotel_email = '' or h.hotel_email is not NULL) then NULL else cast(h.hotel_email as nvarchar(50)) end,
    h.hotel_datains
from [Meridiano].[dbo].[HOTEL] as h
where h.hotel_cit is not null
```

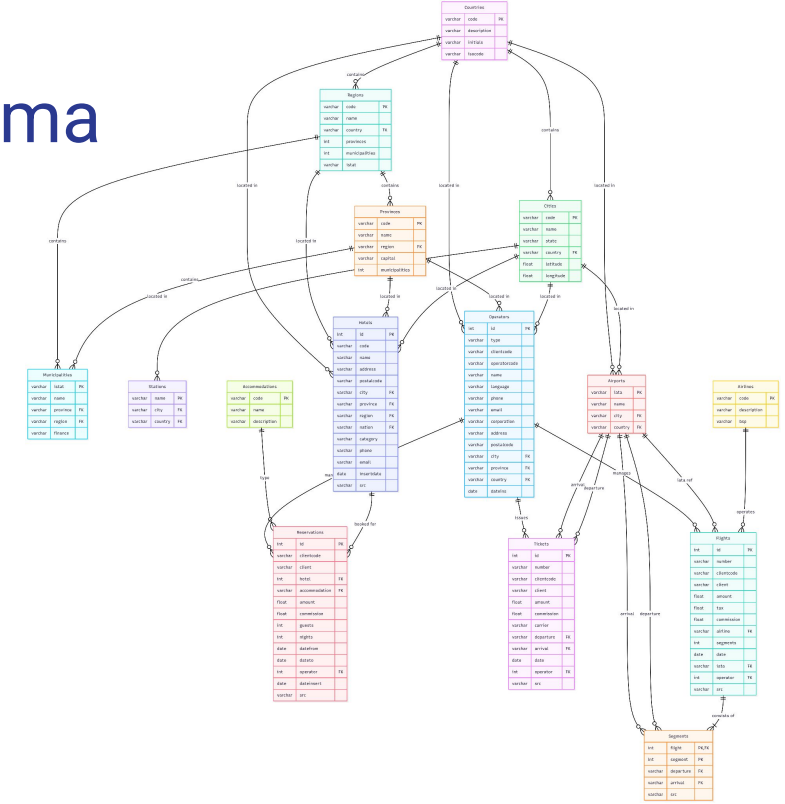
```sql
INSERT INTO reconcileddatalayer.dbo.cities
    (code, name, state, country, latitude, longitude
SELECT
    CAST(c.g_cit_code AS NVARCHAR(3)),
    CAST(c.g_cit_name AS NVARCHAR(50)),
    c.g_cit_statecode,
    c.g_cit_countrycode,
    c.g_cit_latitude,
    c.g_cit_longitude
FROM (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY g_cit_code
            ORDER BY g_cit_name
        ) AS rn
    FROM meridiano.dbo.g_cities
) c
WHERE c.rn = 1
  AND NOT EXISTS (
      SELECT 1
      FROM reconcileddatalayer.dbo.cities rdl
      WHERE rdl.code = c.g_cit_code
  );
```

Reconciled Data Layer

- Data standardization, extraction, cleaning and integration
  - Only meaningful tables and fields kept in the schema
  - Removal of duplicate records and inconsistent data
- Original ER schema slightly modified to gather proper reconciliation and integration
  - Currency uniformation prior to data loading
  - Different tables for flight tickets and train tickets
  - Decoupling between flights and stopover/routing data
  - Omitted representation of packages and categories

Reconciled Data Layer ER Schema





Multidimensional Fact Modeling

- Denormalization as tool to improve query performance
  - Pros: number of table joins needed decreases
  - Cons: data overhead and redundancy increases
- Facts defined as event occurrences or transactions
  - Ticket sales, flights, hotel reservations
- Dimensions identified as everything useful for data aggregation
  - Places, Hotels, Dates, Hubs, Categories
- Resulting model is a “galaxy” of correlated facts

Fact Tables

Made by measurable values and foreign keys linked to dimensions

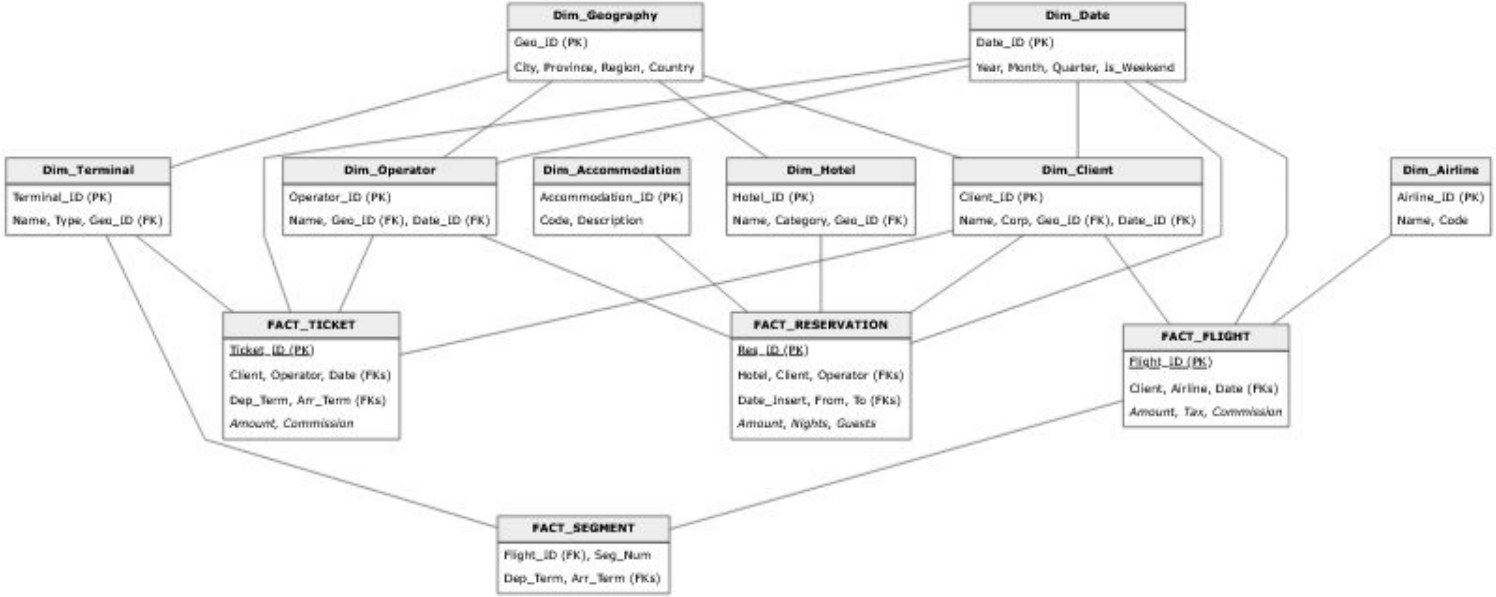
- *Fact Reservation* having measures as *amount*, *guest/night number* to and linked to dimensions like *Hotel*, *Accommodation*, *Date*, *Operator*
- *Fact Ticket* having measures as *amount*, *commission*, *carrier* and linked to dimensions like *Date*, *Client* and *Terminal*
- *Fact Flight/Segment* subdivision between financial/business information and routing/navigation aspects. *Segment* modeled as factless fact, linked to *Fact Flight* and *Dimension Terminal*

Dimension Tables

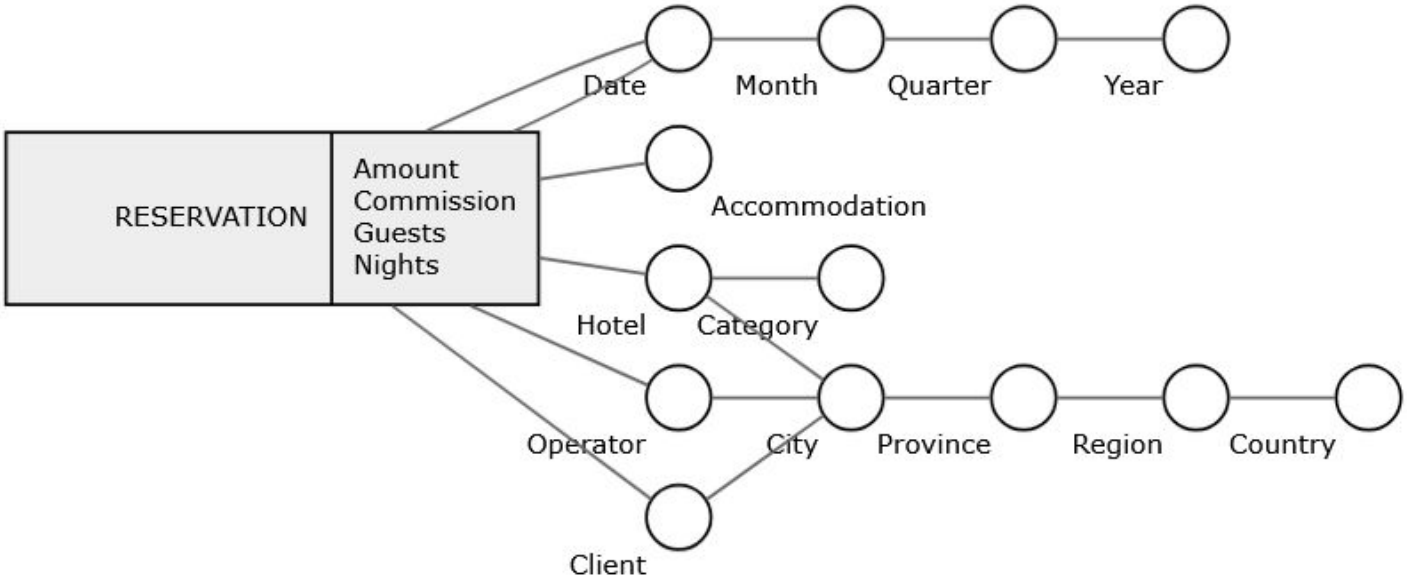
Dimensions having an ID working as foreign key to facts

- *Dimension Date* useful to aggregate facts by their occurrence moment and time interval
- *Dimension Geography* collapsing the geographical hierarchy made by *Cities*, *Municipalities*, *Provinces*, *Regions* and *Countries*
- *Dimension Operator/Client* ambivalent for the corporation insights
- *Dimension Terminal* collapsing all the information about travel hubs
- *Dimensions Airline* and *Accommodation* are self-explanatory

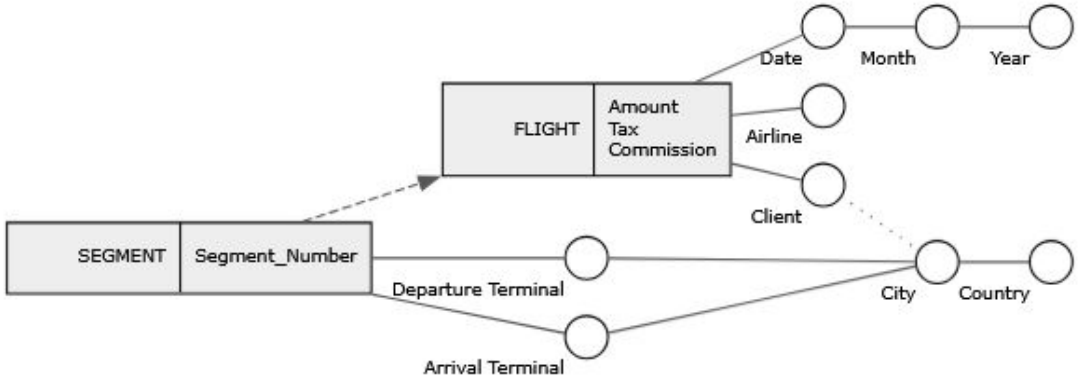
Dimensional Fact Model



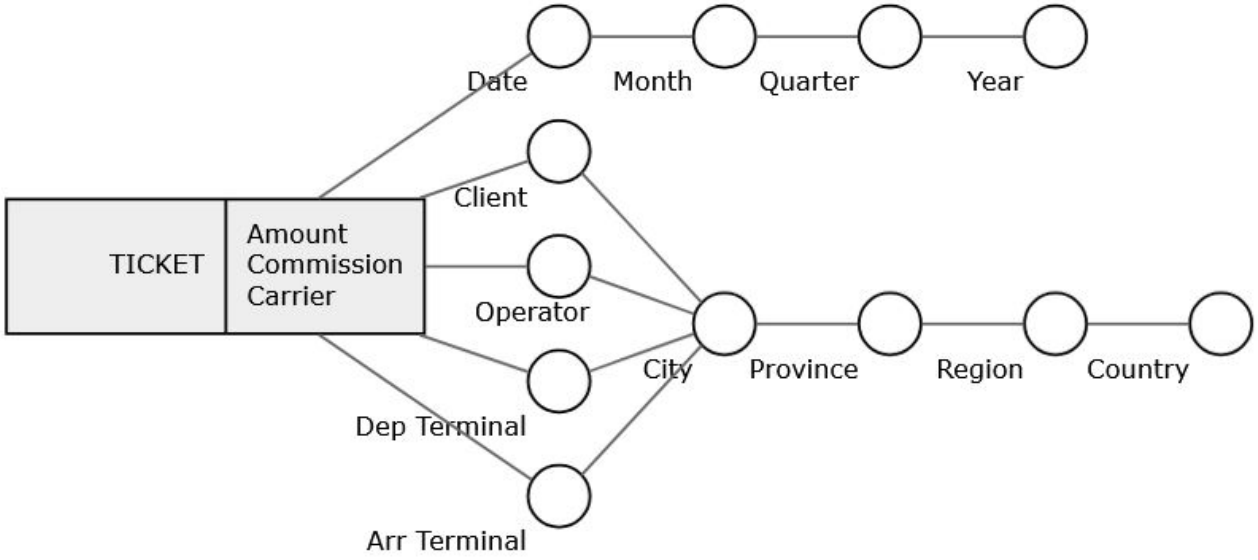
Data Hierarchies



Data Hierarchies



Data Hierarchies

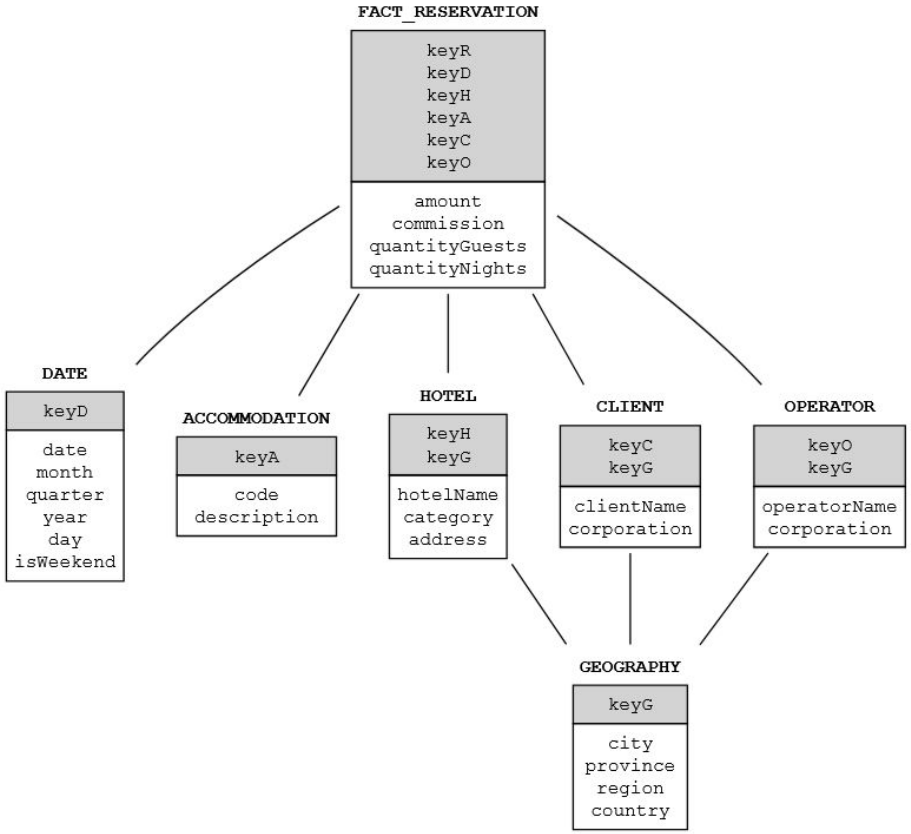


Snowflake Schema Drawbacks

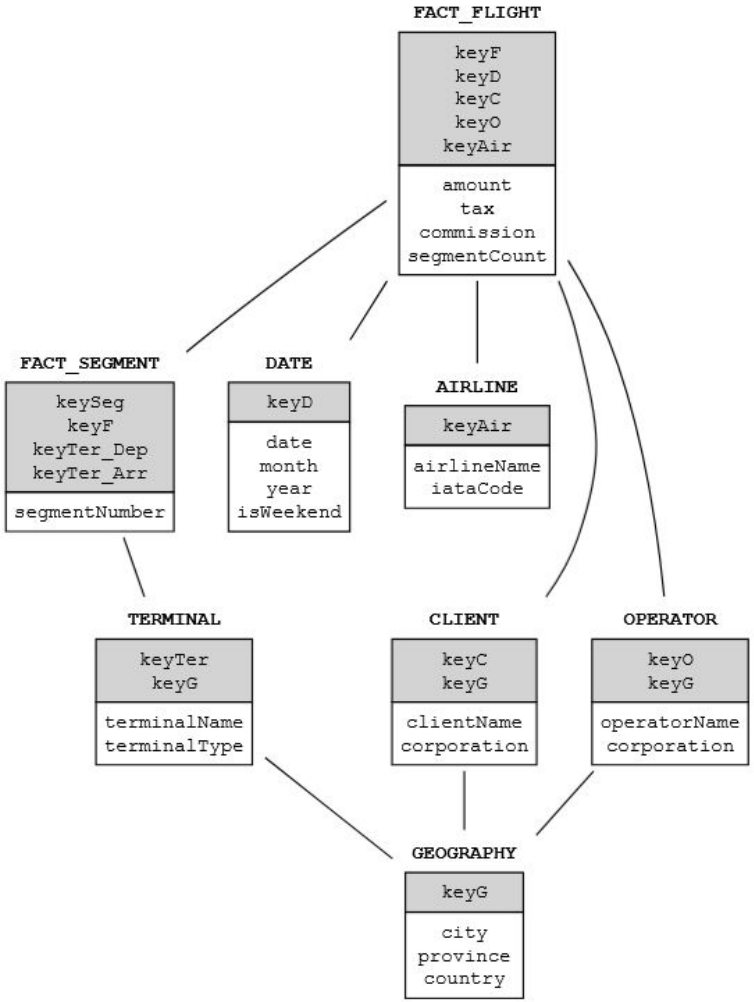
- Presence of shared hierarchies between different dimensions
- Distinction between primary and secondary dimensions
- Longer time needed to query secondary dimensions
- Need for more surrogate keys, tables and joins

Snowflake Schema Advantages

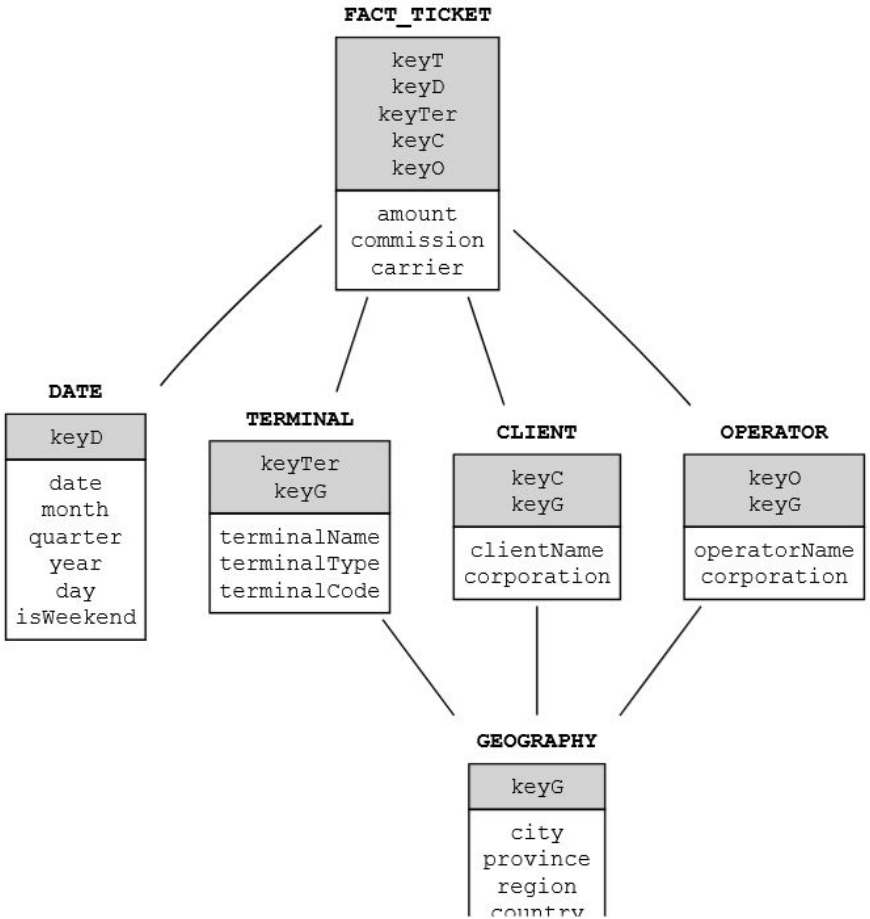
- Consistent with the Dimensional Fact Model
- Data coherence guaranteed
  - any changes only updates few rows
  - risk of misaligned data and anomalies reduced
- Data overhead and redundancy not negligible due to the size of tables
- Many facts but relatively few aggregators, storage occupancy way worse than an extra join
- Drill Down & Roll Up optimized for aggregators
- I evaluated the Star Schema, but due to the highly hierarchical domain nature I preferred the formal cleanliness and referential integrity



Snowflake Schema



Snowflake Schema



Snowflake Schema

Data Marts

Subdivision of between three marts came automatically

- Hotel reservations and relative dimensions
- Train tickets and affiliate
- Flights & Co.

ETL Queries (again)

```sql
USE DimensionalFactModel
GO
CREATE TABLE Dim_Accommodation(
    Accommodation_ID INT IDENTITY(1,1
    Accommodation_Code_BK NVARCHAR(50
    Accommodation_Description NVARCHA
```

```sql
USE DimensionalFactModel
GO
CREATE TABLE Dim_Hotel (
    Hotel_ID INT IDENTITY(1,1) PRIMARY KE
    -- Business Key composed by original
    Source_Hotel_ID NVARCHAR(50),
    Source_System NVARCHAR(50),
    Hotel_Code NVARCHAR(50),
    Hotel_Name NVARCHAR(255),
    Hotel_Address NVARCHAR(255),
    Email NVARCHAR(255),
    Postal_Code NVARCHAR(50),
    Category NVARCHAR(50),
    Phone NVARCHAR(50),
    -- Link to Dim_Geography
    Geography_ID INT,
    -- Link to Dim_Date
    Date_ID INT
);
```

```sql
USE DimensionalFactModel
GO
-- Municipalities insert (Italy only)
INSERT INTO Dim_Geography
SELECT
    'Municipality' AS Location_Type,
    TRIM(m.istat) AS Location_Code_BK,
    TRIM(m.name) AS City_Name,
    TRIM(p.code) AS Province_Code,
    TRIM(p.name) AS Province_Name,
    TRIM(r.code) AS Region_Code,
    TRIM(r.name) AS Region_Name,
    TRIM(c.initials) AS Country_Code_Alpha2,
    TRIM(c.code) AS Country_Code_Alpha3,
    TRIM(c.description) AS Country_Name
FROM [ReconciledDataLayer].[dbo].[Municipalities] m
LEFT JOIN [ReconciledDataLayer].[dbo].[Provinces] p ON TRIM(m.province
LEFT JOIN [ReconciledDataLayer].[dbo].[Regions] r ON TRIM(p.region) =
LEFT JOIN [ReconciledDataLayer].[dbo].[Countries] c ON TRIM(r.country)

-- Cities insert (Region = NULL for non-italian cities)
INSERT INTO Dim_Geography (
    Location_Type, Location_Code_BK, City_Name,
    Province_Code, Province_Name,
    Country_Code_Alpha2, Country_Code_Alpha3, Country_Name
)
SELECT
    'City' AS Location_Type,
    TRIM(ci.code) AS Location_Code_BK,
    TRIM(ci.name) AS City_Name,
    TRIM(ci.state) AS Province_Code,
    TRIM(ci.state) AS Province_Name,
    TRIM(c.initials) AS Country_Code_Alpha2,
    TRIM(c.code) AS Country_Code_Alpha3,
    TRIM(c.description) AS Country_Name
FROM [ReconciledDataLayer].[dbo].[Cities] ci
LEFT JOIN [ReconciledDataLayer].[dbo].[Countries] c ON TRIM(ci.country
```

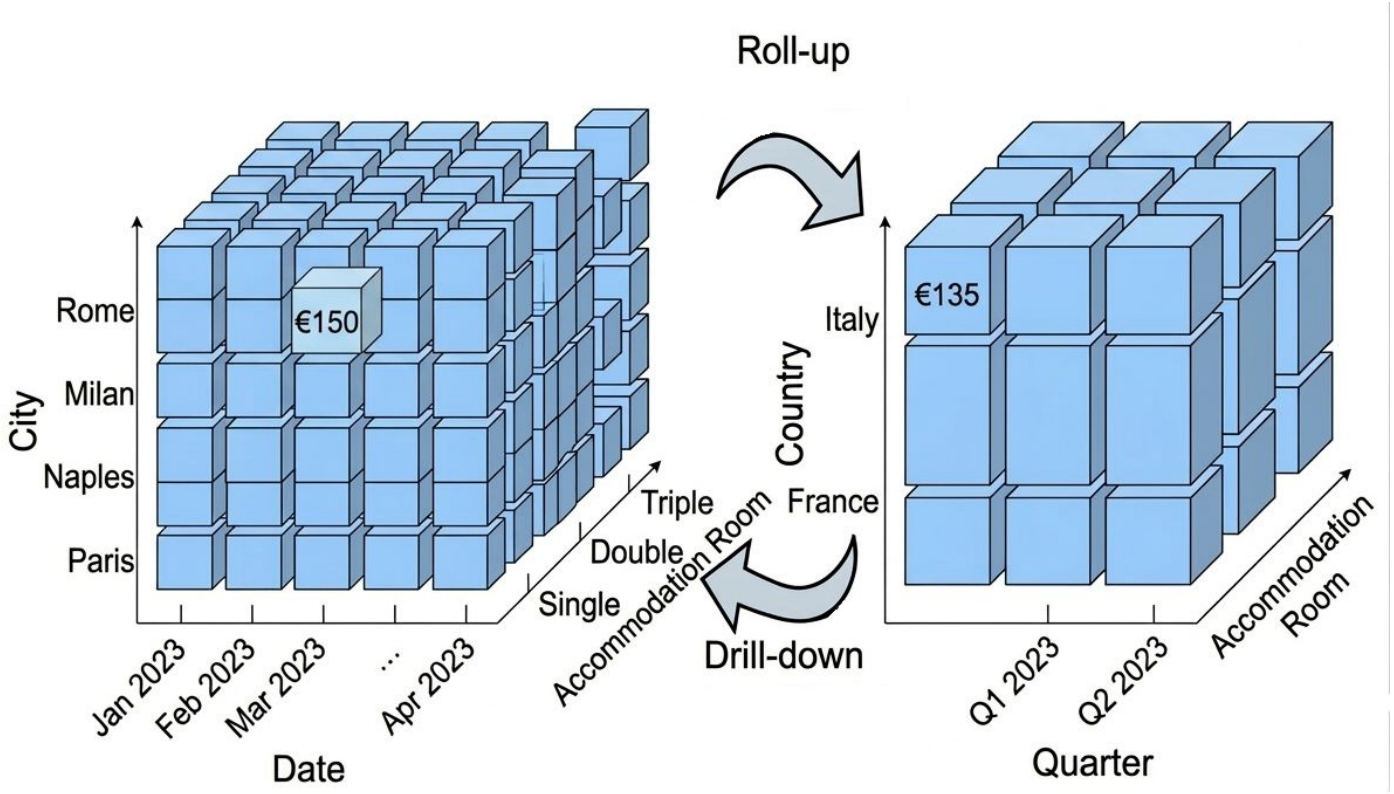
```sql
USE DimensionalFactModel
GO
INSERT INTO Dim_Airline (
    Airline_Code_IATA,
    Airline_Name
)
SELECT
    a.code,
    a.description
FROM ReconciledDataLayer.dbo.Airlines as a
```

```sql
USE DimensionalFactModel
GO
CREATE TABLE Dim_Date (
    -- Formato YYYY/MM/DD
    Date_ID INT PRIMARY KE
    Full_Date DATE NOT NUL
    [Year] INT NOT NULL,
    [Month] INT NOT NULL,
    Month_Name NVARCHAR(50
    [Day] INT NOT NULL,
    Day_Of_Week INT NOT NU
    Day_Name NVARCHAR(50),
    [Quarter] INT NOT NULL
    Is_Weekend BIT NOT NUL
);
```

Dimensional Cube

- Fact occurrence represented as a cube, dimensions represented as *x/y/z* axis
- Granularity as level of detail over dimension axis
- Olap navigation as sequence of operations like *roll-up*, *drill-down*, *slice & dice* etc.
- Following: *Roll-Up* and *Drill-Down* operations between *average amount* for *reservations*
- *x/y/z* axis representing Dimensions *Date*, *Geography* and *Accommodation Type* at different levels of granularity

Dimensional Cube



Olap Session

```sql
USE DimensionalFactModel
GO
-- OLAP step 1: Base Navigation Point + SLICE on Accommodation
-- View: Average price for night, nation and Year (double rooms only)
SELECT
    g.Country_Name,
    d.[Year],
    AVG(f.Reservation_Amount / NULLIF(f.Reservation_Nights, 0)) AS Avg_Price_Per_Night
FROM Fact_Reservation f
JOIN Dim_Hotel h ON f.Reservation_Hotel_ID = h.Hotel_ID
JOIN Dim_Geography g ON h.Geography_ID = g.Geography_ID
JOIN Dim_Date d ON f.Date_From_ID = d.Date_ID
JOIN Dim_Accommodation a ON f.Reservation_Accommodation_ID = a.Accommodation_ID
WHERE a.Accommodation_Description LIKE '%Double%'
GROUP BY
    g.Country_Name,
    d.[Year]
ORDER BY
    g.Country_Name,
    d.[Year];
```

100 % 7 0

Risultati | Messaggi

	Country_Name	Year	Avg_Price_Per_Night
37	AUSTRALIA	2019	120,034
38	AUSTRALIA	2020	113,105
39	AUSTRALIA	2021	180,666666666667
40	AUSTRALIA	2022	173,685833333333
41	AUSTRALIA	2023	168,768253968254
42	AUSTRALIA	2024	170,583181818182
43	AUSTRALIA	2025	89
44	AUSTRIA	2018	91,4498245614035
45	AUSTRIA	2019	154,6175
46	AUSTRIA	2020	79,375
47	AUSTRIA	2021	27,4111182072829
48	AUSTRIA	2022	109,130512820513

Esecuzione della query completata.

Olap Session

```sql
USE DimensionalFactModel
GO
-- STEP 2: DRILL-DOWN (Country -> City, Year -> Quarter) + DICE (Italy)
SELECT
    g.City_Name,
    d.[Year],
    d.[Quarter],
    AVG(f.Reservation_Amount / NULLIF(f.Reservation_Nights, 0)) AS Avg_Price_Per_Night
FROM Fact_Reservation f
JOIN Dim_Hotel h ON f.Reservation_Hotel_ID = h.Hotel_ID
JOIN Dim_Geography g ON h.Geography_ID = g.Geography_ID
JOIN Dim_Date d ON f.Date_From_ID = d.Date_ID
JOIN Dim_Accommodation a ON f.Reservation_Accommodation_ID = a.Accommodation_ID
WHERE a.Accommodation_Description LIKE '%Double%'
  AND g.Country_Name = 'ITALIA'
GROUP BY
    g.City_Name,
    d.[Year],
    d.[Quarter]
ORDER BY
    g.City_Name,
    d.[Year],
    d.[Quarter];
```

100 % 7 0

Risultati Messaggi

	City_Name	Year	Quarter	Avg_Price_Per_Night
305	Cuneo	2024	2	78,4027586206897
306	Cuneo	2024	3	93,3333333333333
307	Cuneo	2024	4	116,764545454545
308	Cuneo	2025	1	95,2
309	Cuneo	2025	2	117
310	Cuneo	2025	3	115
311	Elba Island	2021	2	95
312	Elba Island	2022	2	134,648484848485
313	Elba Island	2023	2	160
314	Firenze	2017	1	42,4166666666667
315	Firenze	2018	1	120,77488372093
316	Firenze	2018	2	114,698918918919

Esecuzione della query completata.

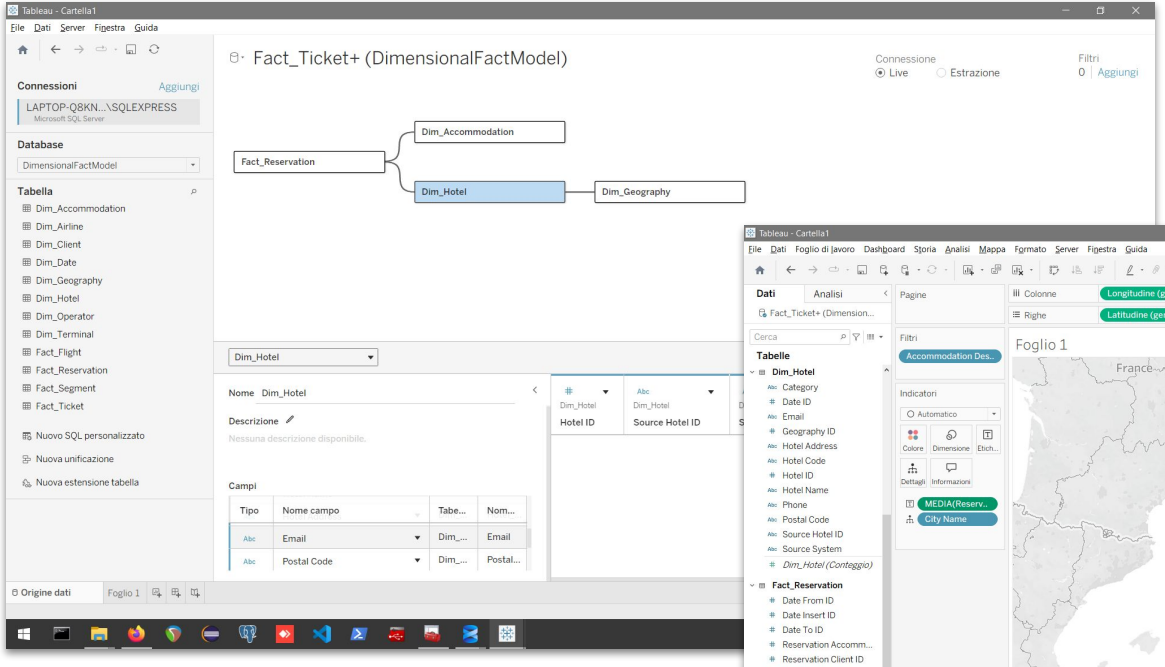
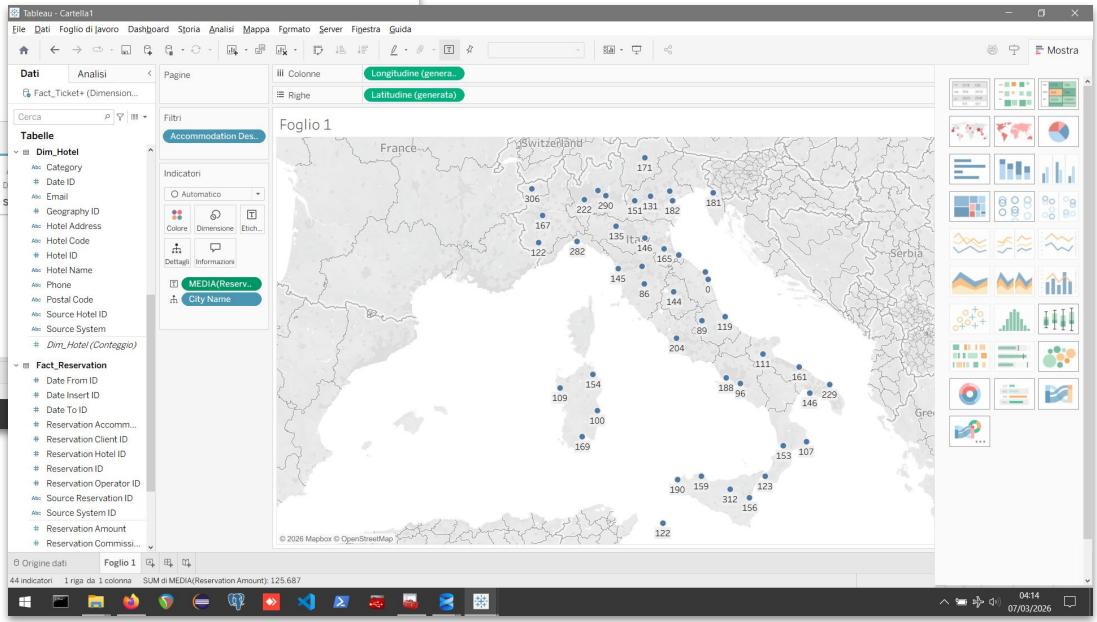
Olap Session

```sql
USE DimensionalFactModel
GO
-- STEP 3: ROLL-UP (Quarter -> Year, City -> Country) + Accommodation comparison
SELECT
    a.Accommodation_Description,
    d.[Year],
    AVG(f.Reservation_Amount / NULLIF(f.Reservation_Nights, 0)) AS Avg_Price_Per_Night
FROM Fact_Reservation f
JOIN Dim_Hotel h ON f.Reservation_Hotel_ID = h.Hotel_ID
JOIN Dim_Geography g ON h.Geography_ID = g.Geography_ID
JOIN Dim_Date d ON f.Date_From_ID = d.Date_ID
JOIN Dim_Accommodation a ON f.Reservation_Accommodation_ID = a.Accommodation_ID
WHERE g.Country_Name = 'ITALIA'
GROUP BY
    a.Accommodation_Description,
    d.[Year]
ORDER BY
    a.Accommodation_Description,
    d.[Year];
```

100 % 7 0

Risultati Messaggi

	Accommodation_Description	Year	Avg_Price_Per_Night
38	Quadruple	2022	96,7333333333333
39	Quadruple	2023	830
40	Single Deluxe	2020	65,5
41	Single superior	2019	1044,44
42	Standard Double	2018	121,578888888889
43	Standard Double	2019	288,630783730159
44	Standard Double	2020	369,123759920635
45	Standard Double	2023	71,3125
46	Standard Double	2024	88,7722007722008
47	Standard Double	2025	45,8536585365854
48	Standard Double for Single ...	2016	7,12746478873239
49	Standard Double for Single	2017	20,7334535010241

Esecuzione della query completata.

Business Intelligence